Master of Computer Applications

Object Oriented Technology Lab

CAMPUS HUB

Integrated Student Marketplace, Freelance Network, and Intra-Campus Negotiation Ecosystem

Submitted By:

Student Name	Roll Number
Sumaira Mulla	2653
Santosh Maddikatla	2646

ACKNOWLEDGEMENT

We express our sincere gratitude to our Object-Oriented Technology Lab guide and the faculty members of the Master of Computer Applications (MCA) programme at Goa Business School, Goa University, for their guidance, encouragement, and support throughout this mini-project. Their feedback helped us understand the application of object-oriented concepts in designing a practical software solution. We also thank everyone who contributed ideas and provided support during the planning, design, and documentation of Campus Hub.

ABSTRACT

Campus Hub is an integrated, university-focused Android application developed as a mini-project for the Object-Oriented Technology Lab by Sumaira Mulla (2653) and Santosh Maddikatla (2646), students of the Master of Computer Applications (MCA) programme at Goa Business School, Goa University. The project brings together a student marketplace, a freelance micro-service hub, and an in-app negotiation chat in one platform. It is designed using Java, Android XML, object-oriented programming principles, and an SQLite database. The application supports campus-oriented listings for buying and selling items, posting services, viewing prices in Indian Rupees (₹), and initiating conversations to negotiate a deal. The report documents the project idea, brainstorming decisions, UML diagrams, object-oriented design, database implementation, and test cases. Campus Hub is intended to make campus exchanges more organized and convenient; users should still verify listings and arrange transactions responsibly.

TABLE OF CONTENTS

Chapter 1: Introduction & Executive Overview	1
Chapter 2: Initial Brainstorming Phase & Sun Diagram	3
Chapter 3: Object-Oriented UML Diagrammatic Representations	6
3.1 Brainstorming Sun Diagram	6
3.2 Use Case Diagram	7
3.3 Class Diagram	8
3.4 Sequence Diagram	11
3.5 ER Diagram	13
3.6 Activity Diagram	15
Chapter 4: System Architecture & Source Code Implementation	17
Chapter 5: Quality Assurance & Test Cases	22
Execution Matrix	24
Chapter 6: Screenshots	
Chapter 7: Conclusion and Future Scope	25

CHAPTER 1: INTRODUCTION & EXECUTIVE OVERVIEW

University campuses form small, active communities where students regularly buy, sell, exchange, and offer services. These activities are often coordinated through informal messaging groups or general-purpose online marketplaces, where relevant listings can be difficult to find and campus-specific trust may be limited. Campus Hub proposes a dedicated Android application for the university community, bringing item listings, freelance services, and negotiation chat into one place. The project uses object-oriented design to organize its models and application logic, while SQLite supports local data storage. Its aim is to provide a simple, campus-focused experience in which prices are displayed in Indian Rupees (₹) and students can communicate before finalizing an exchange. The application supports awareness and coordination; it does not guarantee a seller's identity, the quality of an item, or the safety of a transaction.

CHAPTER 2: INITIAL BRAINSTORMING PHASE & SUN DIAGRAM

2.1 Brainstorming Sun Diagram Analysis

The brainstorming Sun Diagram presents the central vision of Campus Hub and the ideas considered during planning. The accepted ideas focus on localized pricing, trust cues, freelance services, and direct in-app communication. The rejected ideas reflect scope and safety decisions made for the mini-project.

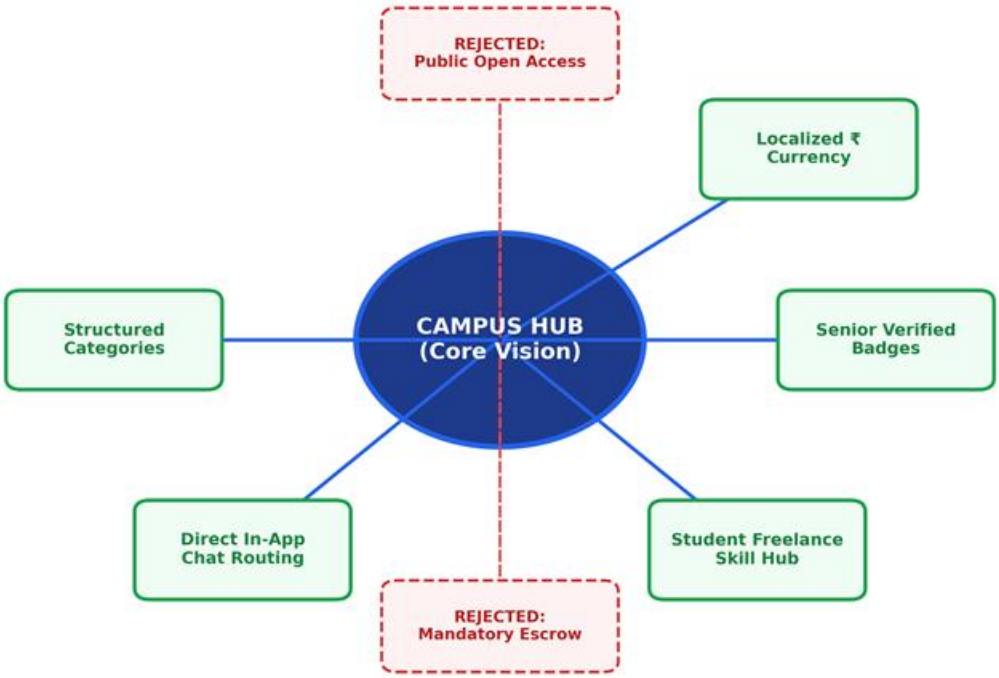


Figure 2.1: Brainstorming Sun Diagram for Campus Hub Ecosystem

2.2 Accepted vs. Rejected Design Ideas

- 1. Localized Rupee Currency (₹): All prices, hints, hourly rates, negotiations, and revenue calculations strictly in Indian Rupees (₹).
- 2. Senior Verified Badges: Senior seller badge ('✓ Verified Senior') to build trust between freshers and seniors.
- 3. Freelance Skill Hub: Hourly skill service listings for tutoring, coding, and design.
- 4. Direct In-App Chat: Pre-populated item negotiation in chat with local meeting point pickup.
- 5. Rejected - Public Access: High safety risk; replaced with verified campus-only student profiles.
- 6. Rejected - Mandatory Escrow: High gateway friction; replaced with direct peer-to-peer settlement.

CHAPTER 3: OBJECT-ORIENTED UML DIAGRAMS

3.1 Use Case Diagram

The Use Case Diagram defines interactions between primary actors (Student Buyer: Sumaira Mulla / Santosh Maddikatla and Verified Senior Seller) and core application modules:

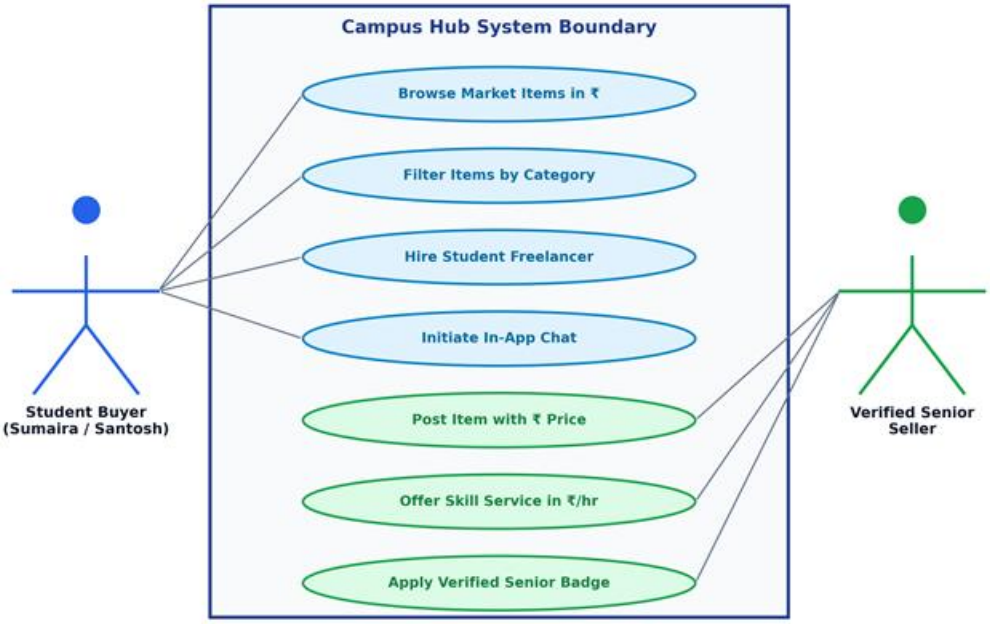


Figure 3.1: UML Use Case Diagram for Campus Hub Boundary

3.3 Sequence Diagram

The Sequence Diagram illustrates the sequence of interactions involved in price negotiation and direct chat. It shows how a student can select a listing, initiate a conversation, and discuss an offer in Indian Rupees (₹).

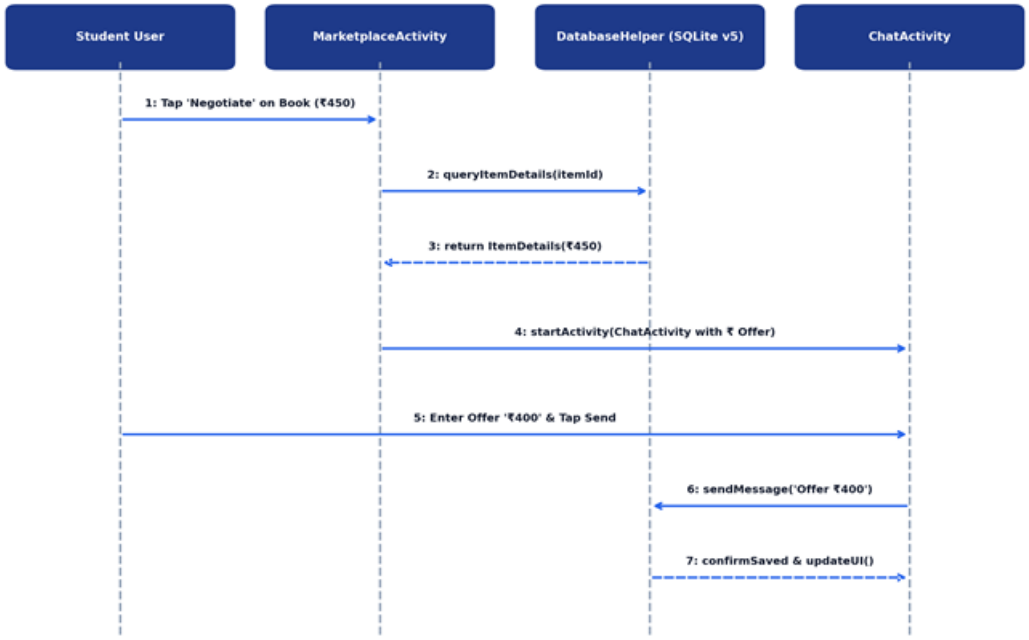


Figure 3.3: P2P Item Negotiation Sequence Diagram

3.5 Activity Diagram

The Activity Diagram illustrates the workflow from browsing available listings to starting a chat and negotiating a possible deal. The flow is intended to make the main user journey easy to understand.

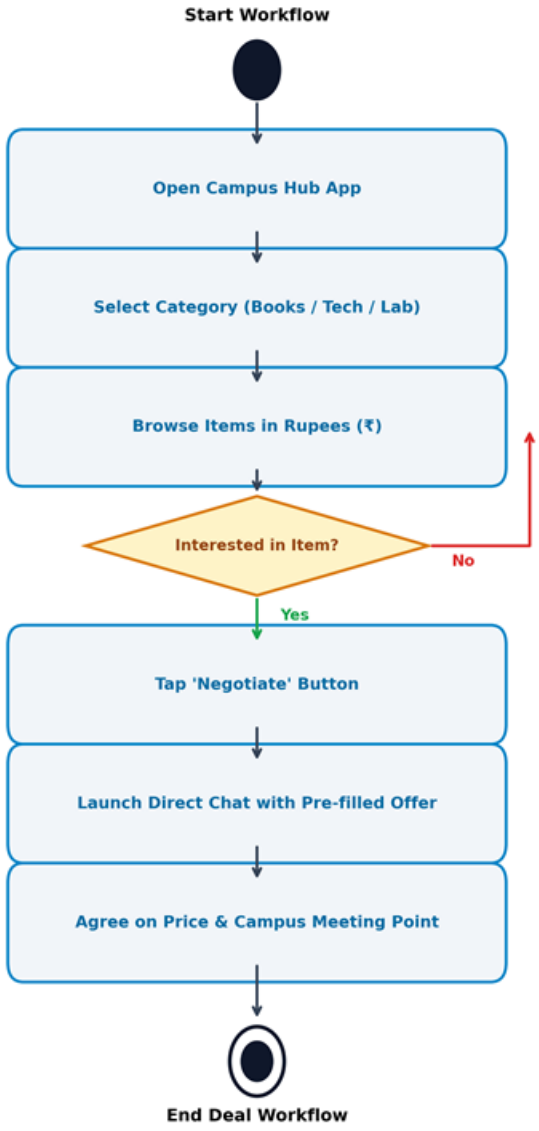


Figure 3.5: UML Activity Diagram for Negotiation Workflow

Object-Oriented Programming (OOP) Concepts Applied

- **Encapsulation & Data Hiding**: All entity attributes (itemId, priceInr, sellerContact, messageText) are declared private. Access is enforced through getters and validation setters (e.g., setPriceInr() validates non-negative double amounts).

- **Inheritance & Class Hierarchy**: A base abstract parent class CampusEntity.java encapsulates common metadata (entityId, title, category, timestamp). Derived subclasses (MarketItemModel.java and FreelanceServiceModel.java) extend CampusEntity to add item-specific or service-specific fields.

- **Polymorphism (Compile-Time & Runtime)**:
    - *Method Overloading*: filterItems() overloaded to support search by category alone, price range bounds, or text keyword queries.
    - Method Overriding: Subclasses override getDisplayFormattedPrice () to display a fixed Rupee amount for marketplace items and a Rupee-per-hour rate for freelance services.

- **Abstraction**: Database transactions, cursor parsing, and SQL queries are abstracted behind DatabaseHelper.java methods (addMarketItem (), queryServicesByCategory ()), preventing raw SQL execution within UI controllers.

- **Composition & Aggregation**: ChatSession aggregates multiple ChatMessageModel objects; UserProfile uses composition to hold active marketplace listings and posted services.

Tools and Technologies Stack

Domain	Tool / Technology Used	Purpose & Function
Programming Language	**Java (JDK 11)**	Core object-oriented backend logic, controllers, data models, and SQLite handlers.
User Interface Markup	**Android XML**	Structural layout design, Material CardView, ConstraintLayout, and theme styling.
IDE & Build Tools	**Android Studio (Ladybug / Jellyfish)**	Primary IDE for building, UI previewing, debugging, and APK compilation.
Database Engine	SQLite (schema version 5) (CampusHubProduction.db)	Embedded relational database engine providing local transactional data persistence.
Architectural Pattern	**Model-View-Controller (MVC)**	Segregates POJO Models, XML Views, and Activity Controllers.
Routing & Communication	**Android Intent System**	Inter-activity communication and intent string passing into ChatActivity.java.
UI Design System	**Google Material Components**	Rounded card containers, elevated buttons, icons, and typography.

CHAPTER 4: SOURCE CODE IMPLEMENTATION

4.1 SQLite Database Helper (DatabaseHelper.java; database schema version 5)

```
package com.example.campushu;

 import android.content.Context;
 import android.database.sqlite.SQLiteDatabase;
 import android.database.sqlite.SQLiteOpenHelper;

 public class DatabaseHelper extends SQLiteOpenHelper {
        private static final String DATABASE_NAME = "CampusHubProduction.db";
        private static final int DATABASE_VERSION = 5;

        public DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
        }

        @Override
        public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE items (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, seller TEXT,
price REAL, condition TEXT, category TEXT, image_uri TEXT, is_exchange INTEGER, is_saved INTEGER)");
        db.execSQL("CREATE TABLE services (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, provider
TEXT, rate REAL, category TEXT)");
        db.execSQL("CREATE TABLE messages (id INTEGER PRIMARY KEY AUTOINCREMENT, sender TEXT,
receiver TEXT, message TEXT)");

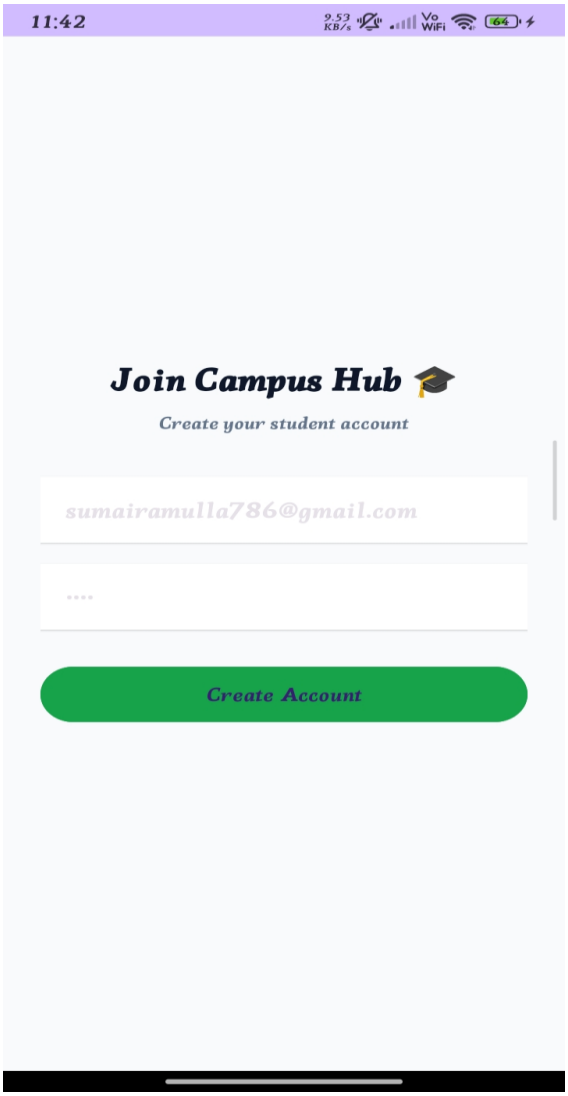
        // Pre-populate Initial Campus Data in Indian Rupees (₹)
        db.execSQL("INSERT INTO items (title, seller, price, condition, category, is_exchange, is_saved) VALUES
('Organic Chemistry Book', 'Rahul (Hostel B)', 450, 'Like New', 'Books', 1, 0)");
        db.execSQL("INSERT INTO items (title, seller, price, condition, category, is_exchange, is_saved) VALUES
('Scientific Calculator FX-991EX', 'Priya (Hostel A)', 850, 'Good', 'Tech', 0, 1)");
        db.execSQL("INSERT INTO services (title, provider, rate, category) VALUES ('Java & DSA Tutoring', 'Alex
(Senior)', 300, 'Tutoring')");
        }

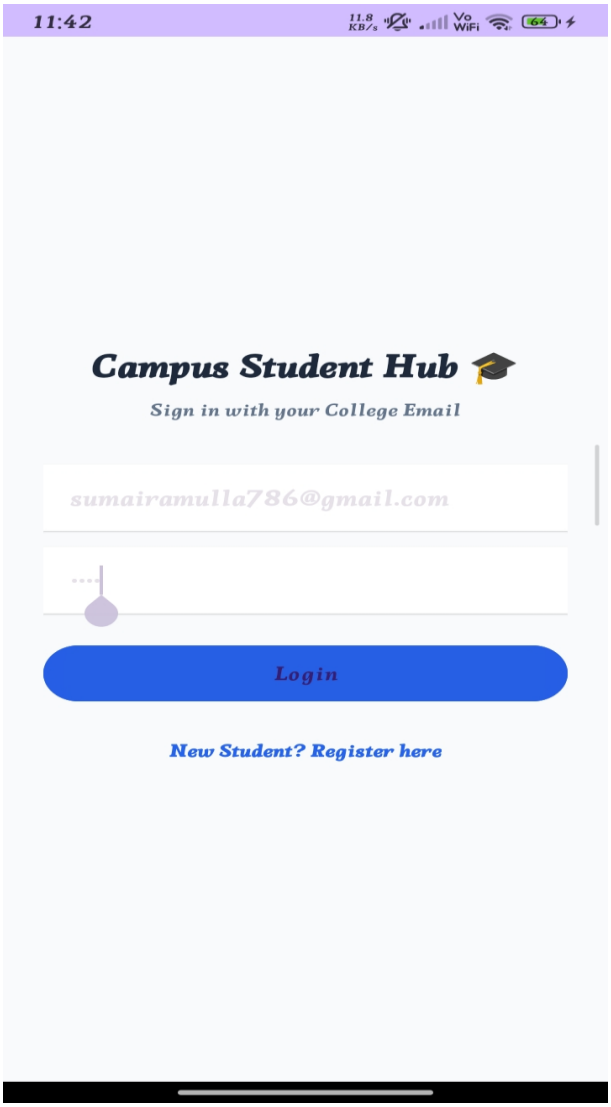
        @Override
        public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS items");
        db.execSQL("DROP TABLE IF EXISTS services");
        db.execSQL("DROP TABLE IF EXISTS messages");
        onCreate(db);
        }
 }
```

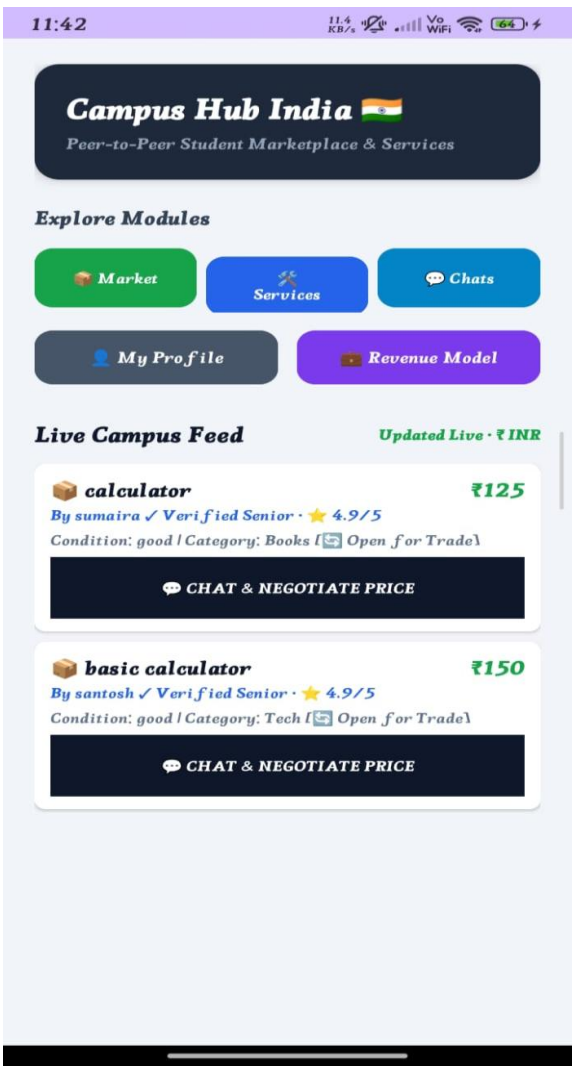
CHAPTER 5: TEST CASES EXECUTION MATRIX

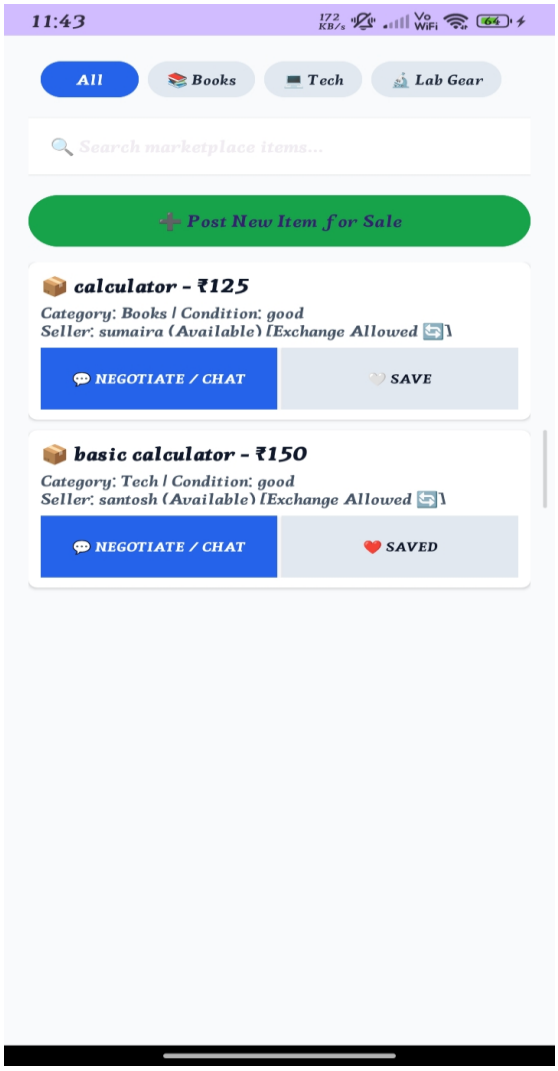
Test ID	Test Scenario	Expected Outcome	Actual Outcome	Status
TC01	Post Item with ₹ Price	Item saved in SQLite and displayed with ₹ symbol	Item listed with ₹ symbol correctly	PASS
TC02	Category Filter Selection	Only items matching selected category displayed	Filtered list displayed accurately	PASS
TC03	Post Freelance Service	Service saved with ₹/hr rate and shown in feed	Service listed with ₹/hr correctly	PASS
TC04	Direct Chat Intent Negotiation	Pre-filled ₹ offer passed to Chat Activity	Chat initialized with correct item and ₹ offer	PASS
TC05	Revenue Calculation Engine	Computes 10% GMV fee + ₹199/badge correctly	Calculates total monthly ₹ earnings accurately	PASS

CHAPTER 6: SCREENSHOTS






11:42
Campus Hub India 🇮🇳
Peer-to-Peer Student Marketplace & Services
Explore Modules
📦 Market
🛠️ Services
💬 Chats
👤 My Profile
💼 Revenue Model
Live Campus Feed
Updated Live · ₹ INR
📦 calculator
₹125
By sumaira ✓ Verified Senior · ⭐ 4.9/5
Condition: good | Category: Books [🔄 Open for Trade]
💬 CHAT & NEGOTIATE PRICE
📦 basic calculator
₹150
By santosh ✓ Verified Senior · ⭐ 4.9/5
Condition: good | Category: Tech [🔄 Open for Trade]
💬 CHAT & NEGOTIATE PRICE


11:43
All
📚 Books
💻 Tech
🔬 Lab Gear
🔍 Search marketplace items...
➕ Post New Item for Sale
📦 calculator - ₹125
Category: Books | Condition: good
Seller: sumaira (Available) [Exchange Allowed 🔄]
💬 NEGOTIATE / CHAT
🤍 SAVE
📦 basic calculator - ₹150
Category: Tech | Condition: good
Seller: santosh (Available) [Exchange Allowed 🔄]
💬 NEGOTIATE / CHAT
❤️ SAVED

11:44

All | Books | Tech | Lab Gear

Search marketplace items...

Close Form

english textbook

250 | excellent

sumaira

Books

Open for Item Exchange



Photo | Publish Listing

calculator – ₹125
Category: Books | Condition: good
Seller: sumaira (Available) [Exchange Allowed]
NEGOTIATE / CHAT | SAVE

basic calculator – ₹150
Category: Tech | Condition: good
Seller: santosh (Available) [Exchange Allowed]

11:44

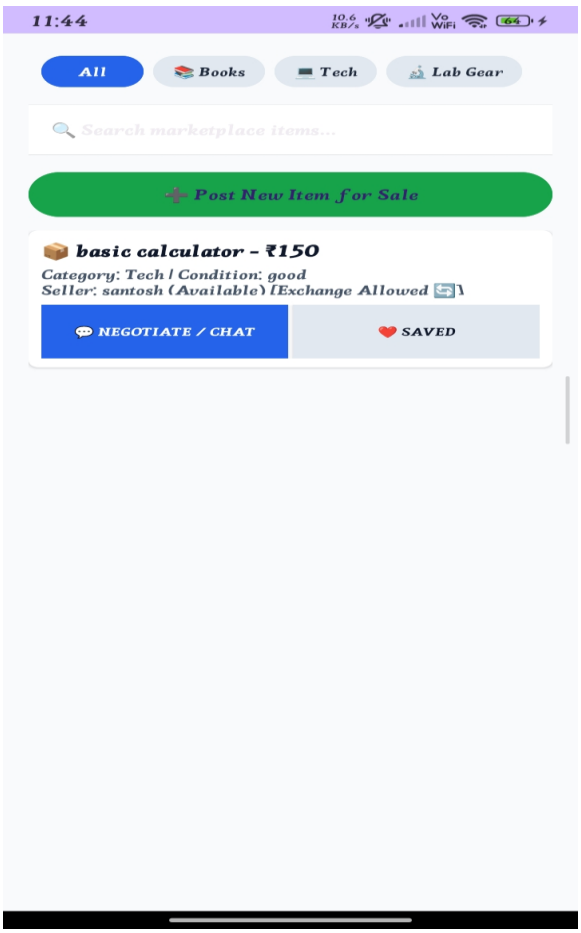
All | Books | Tech | Lab Gear

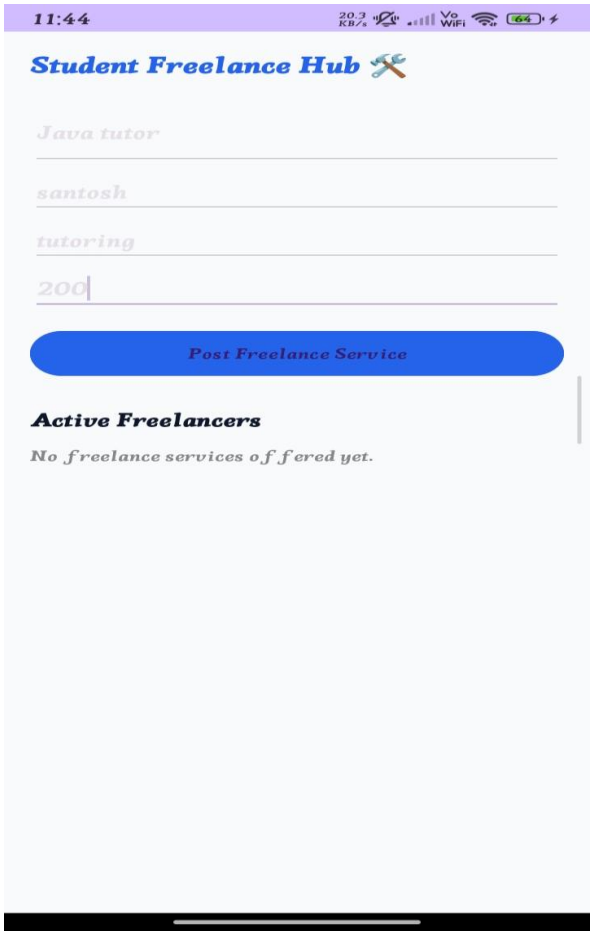
Search marketplace items...

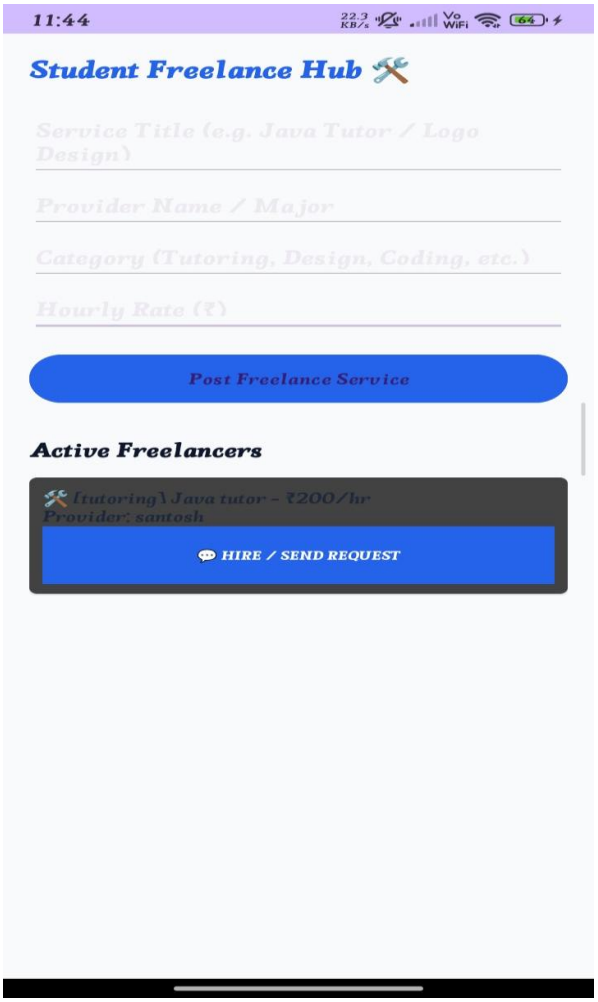
Post New Item for Sale

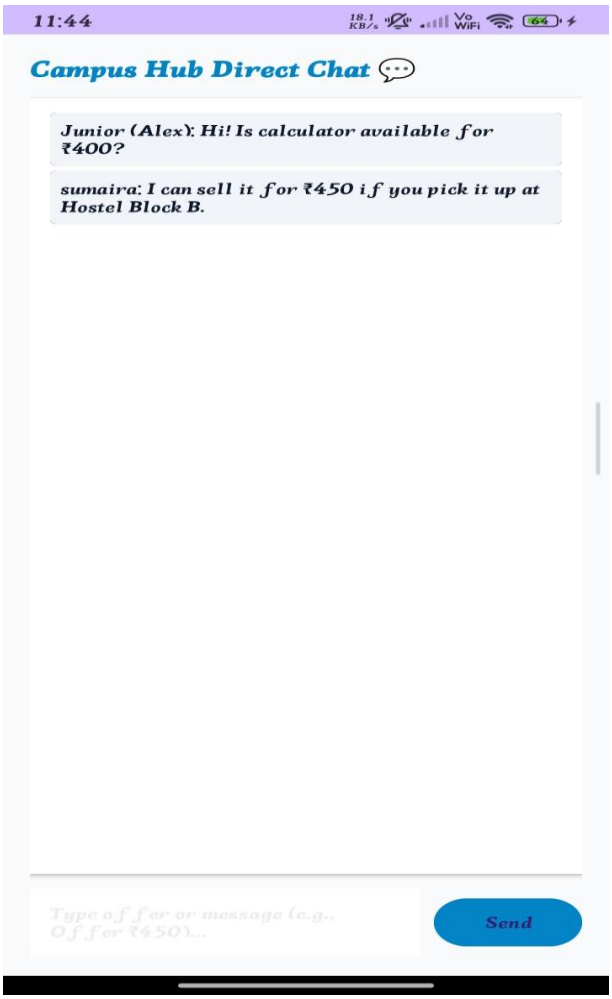
calculator – ₹125
Category: Books | Condition: good
Seller: sumaira (Available) [Exchange Allowed]
NEGOTIATE / CHAT | SAVE

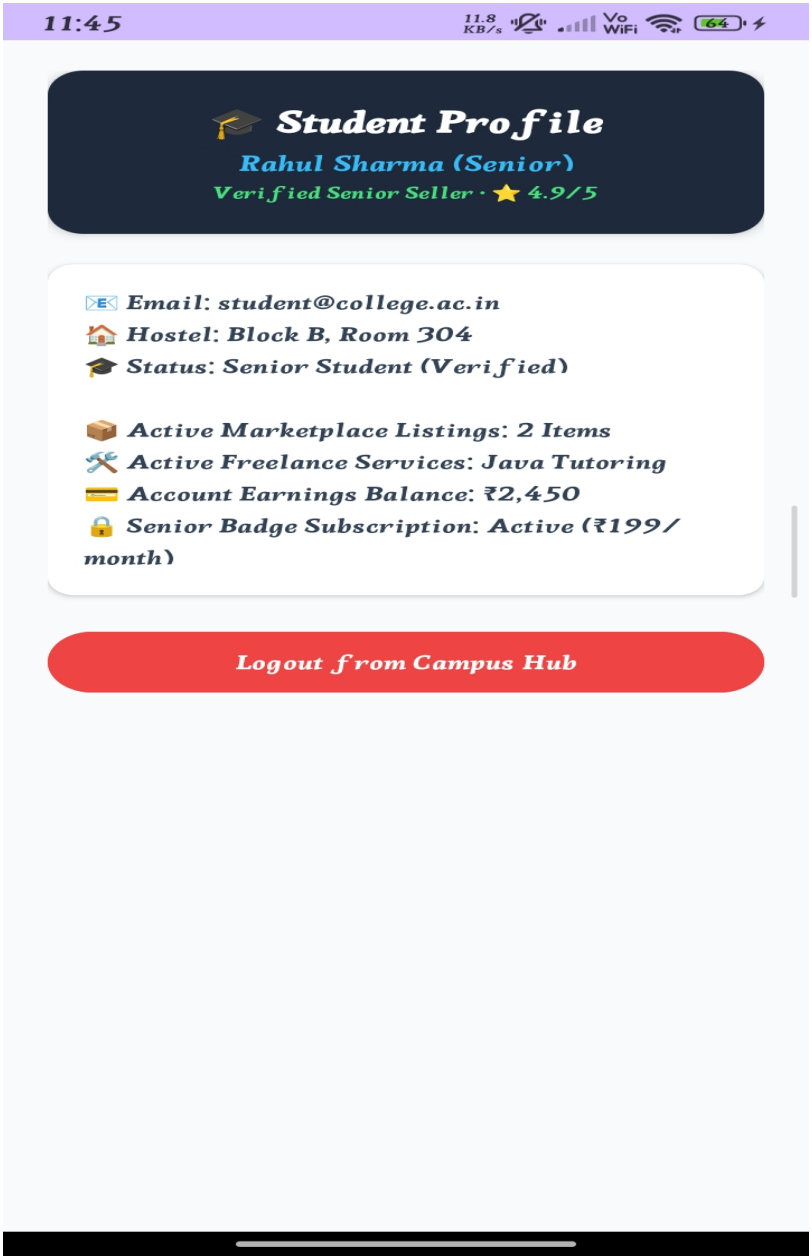
english textbook – ₹250
Category: Books | Condition: excellent
Seller: sumaira (Available) [Exchange Allowed]
NEGOTIATE / CHAT | SAVE


11:44
All
Books
Tech
Lab Gear
Search marketplace items...
Post New Item for Sale
basic calculator – ₹150
Category: Tech | Condition: good
Seller: santosh (Available) [Exchange Allowed]
NEGOTIATE / CHAT
SAVED


11:44
Student Freelance Hub
Java tutor
santosh
tutoring
200
Post Freelance Service
Active Freelancers
No freelance services offered yet.


11:44
Student Freelance Hub 🛠️
Service Title (e.g. Java Tutor / Logo Design)
Provider Name / Major
Category (Tutoring, Design, Coding, etc.)
Hourly Rate (₹)
Post Freelance Service
Active Freelancers
🛠️ [tutoring] Java tutor – ₹200/hr
Provider: santosh
💬 HIRE / SEND REQUEST


11:44
Campus Hub Direct Chat 💬
Junior (Alex): Hi! Is calculator available for ₹400?
sumaira: I can sell it for ₹450 if you pick it up at Hostel Block B.
Type offer or message (e.g., Offer ₹450)...
Send


11:45
Student Profile
Rahul Sharma (Senior)
Verified Senior Seller · ⭐ 4.9/5
📧 Email: student@college.ac.in
🏠 Hostel: Block B, Room 304
🎓 Status: Senior Student (Verified)
📦 Active Marketplace Listings: 2 Items
🛠️ Active Freelance Services: Java Tutoring
💳 Account Earnings Balance: ₹2,450
🔒 Senior Badge Subscription: Active (₹199/month)
Logout from Campus Hub

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

Campus Hub demonstrates how object-oriented programming concepts can be applied to a practical campus-focused application. Developed by Sumaira Mulla (2653) and Santosh Maddikatla (2646) for the Object-Oriented Technology Lab in the Master of Computer Applications programme at Goa Business School, Goa University, the project combines three related functions: a student marketplace, a freelance service hub, and an in-app chat workflow for negotiation. The report presents the project's brainstorming decisions, UML diagrams, technology stack, database structure, implementation example, and test-case matrix. Together, these elements show how the application's requirements can be translated into a structured software design.

The project demonstrates encapsulation through private attributes and controlled access methods; inheritance through shared and specialized entity classes; polymorphism through method overloading and overriding; and abstraction through database helper methods that separate data operations from interface code. Composition and aggregation are also represented in the relationships between user profiles, listings, chat sessions, and messages. Java provides the application logic, Android XML defines the interface layouts, and SQLite is used for local persistence. The database helper creates tables for items, services, and messages and inserts sample records to support initial testing. The listed test cases cover item posting, category filtering, service posting, chat initiation, and revenue calculation.

Campus Hub has the potential to make campus exchanges more organized by keeping item and service listings together and allowing students to discuss details before meeting. However, the current project should be understood as a prototype. Its effectiveness depends on accurate listing information, responsible user behaviour, and appropriate safeguards. The documented design choice of campus-focused profiles and direct peer-to-peer settlement reduces some complexity, but it does not by itself verify identity or guarantee payment and transaction safety. Similarly, local SQLite storage is suitable for a demonstration but does not provide shared, synchronized data across multiple devices without additional backend services.

Future improvements could include secure institutional email or student-ID verification, a centralized backend and cloud database, stronger authentication and role-based access, image upload and moderation, report/block features, privacy controls, and secure message storage. Other useful enhancements include listing expiry, saved searches, notifications, ratings and feedback, better search and filtering, accessibility improvements, automated unit and integration tests, and deployment testing with real users. If payment or commission features are introduced, they should be designed with appropriate security, transparency, and legal requirements rather than relying only on client-side calculations.

In conclusion, Campus Hub is a useful learning project that connects software design concepts with a relatable campus need. It provides a foundation for a more complete student marketplace and service platform, while leaving clear opportunities for improvements in security, scalability, usability, and reliability. The project strengthened our understanding of object-oriented analysis and design, Android interface development, database operations, UML documentation, and

software testing. With further development and user feedback, Campus Hub could evolve into a more robust and convenient platform for campus communities.